

Modül 08: Record'lar ve Enum'lar

Modül 06 size `equals`, `hashCode` ve `toString` metotlarını elle yazdırdı ve yanlış yaptığınızda bir `HashSet` kümesinin içeriğini kaybettiğini gösterdi. Bu modül o problemin cevabı.

Record'lar veri taşır. Enum'lar örnek kümesini sabitler. İkisi birlikte, eski Java'nın meşhur olduğu elle yazılan tekrar kodunun büyük bölümünü ortadan kaldırır.

Neler Öğreneceksiniz

- Bir record'un ne ürettiği, iddia edilerek değil `javap` ile gösterilerek
- Doğrulama ve normalleştirme için kompakt kurucular
- Bir record için "değişmez" sözcüğünün kulağa geldiğinden neden daha sığ olduğu
- Durum ve davranış taşıyan, sabit örnekli sınıflar olarak enum'lar
- `ordinal()` metodunun neden bir tuzak olduğu
- Sınıf, record ve enum arasında nasıl seçim yapılacağı

Başka Bir Dilden Geliyorsanız

	Python	Java
Veri taşıyıcı	<code>@dataclass</code> ya da <code>NamedTuple</code>	<code>record</code>
Eşitlik	dekoratör üretir	record üretir
Sabit örnek kümesi	<code>enum.Enum</code>	<code>enum</code> , ama her sabit tam bir nesne
Sabit başına davranış	zahmetli	her sabitin uyguladığı soyut metot
Değişmezlik	<code>frozen=True</code> , yine de sığ	record'lar, yine de sığ

Sıgılık uyarısı her iki dilde de geçerli ve aynı sebepten. Değiştirilebilir bir liste tutan dondurulmuş bir kap, hâlâ değiştirilebilir bir liste tutar.

Ders

Record nedir

```
record Point(int x, int y) { }
```

O tek satır size bir kurucu, `equals`, `hashCode`, `toString` ve bileşen başına bir erişimci verir. `Records.java` dosyasından:

```
toString: Point[x=3, y=4]
equals:    true
hashCode: true
set size: 1
```

Son satır asıl mesele. Modül 06'nın elle yazılmış `BadPoint` sınıfı iki eşit nesneyi bir `HashSet` içine koydu ve sonra ikisini de bulamadı. Bir record o hatayı yapamaz.

Neyin üretildiğini görmek için Modül 01'deki tekniği kullanın. `public record Money(long cents, String currency) { }` için:

```
public final class Money extends java.lang.Record {
    public Money(long, java.lang.String);
    public final java.lang.String toString();
    public final int hashCode();
    public final boolean equals(java.lang.Object);
    public long cents();
    public java.lang.String currency();
}
```

Bundan üç şey çıkar:

- `final class` — record'lar asla genişletilemez.
- `extends Record` — her record'un zaten bir üst sınıfı vardır, dolayısıyla başka bir şeyi genişletemez. Arayüzleri serbestçe uygulayabilir.
- **Erişimcilerde `get` öneki yoktur.** `p.getX()` değil `p.x()`.

Bir record ek metotlar, statik fabrikalar ve arayüzler içerebilir. İçeremeyeceği şey **ek örnek durumudur**:

```
record Bad(int x) { private int cached; } // does not compile
```

Bu kısıt garantinin kendisidir. Bir record *bileşenlerinin ta kendisidir*, dolayısıyla eşit bileşenli iki record birbirinin yerine geçer.

Kompakt kurucular

Bir record'un doğrulama ve normalleştirme yaptığı yer. `CompactConstructors.java` dosyasından:

```
record Email(String address) {
    Email {
        if (address == null || !address.contains("@")) {
            throw new IllegalArgumentException("not an email address: " + address);
        }
        address = address.strip().toLowerCase();
    }
}
```

Parametre listesi yok, alan ataması yok. Derleyici parametreleri bu gövde çalıştıktan **sonra** alanlara atar, dolayısıyla parametreyi yeniden atamak neyin saklanacağını değiştirir. Burada `this.address = ...`

yazmak derleme hatasıdır.

Normalleştirme görüldüğünden daha önemli. Bir record'un `equals` metodu bileşenleri karşılaştırır, dolayısıyla mantıksal olarak eşit iki değer farklı bileşenlere sahip olabiliyorsa record yalan söylüyordur:

```
new Fraction(6, 8)    -> 3/4
new Fraction(-1, -2) -> 1/2
```

Kurucuda sadeleştirme ve işaret normalleştirilmesi olmasaydı `6/8` ile `3/4` aynı sayı oldukları hâlde eşitsiz olurdu. **Bir record ham bir demet yerine bir kavramı temsil ettiğinde, kompakt kurucuda normalleştirin.**

Sığ değişmezlik

Bir record'un alanları `final` olur. Dilin söz verdiği tek şey bu. [ShallowImmutability.java](#) dosyasından:

```
--- the leak ---
built with: Leaky[title=post, tags=[draft]]
after the caller edited their own list: Leaky[title=post, tags=[draft, smuggled-in]]
after editing the accessor's result:    Leaky[title=post, tags=[draft, smuggled-in,
smuggled-out]]

--- sealed off ---
built with: Safe[title=post, tags=[draft]]
after the caller edited their own list: Safe[title=post, tags=[draft]]
the accessor's list refused modification
```

Tek satırla kapatılan iki ayrı sızıntı:

```
record Safe(String title, List<String> tags) {
    Safe {
        tags = List.copyOf(tags);
    }
}
```

`List.copyOf` kopyalar, böylece çağıranın listesi artık sizin değildir, ve değişmez bir liste döndürür, böylece erişimci onu doğrudan vermekte serbesttir.

Bu record'lar için sıradan sınıflardan daha çok önemlidir, çünkü record'lar tam olarak `equals` metotları bedava geldiği için harita anahtarı ve küme elemanı olarak kullanılır. Modül 06'nın mutasyon tuzağı burada tümüyle geçerli: bir `Leaky` nesnesini `HashSet` içine koyun, listesini değiştirin, nesne onu hâlâ sayan kümenin içinde kaybolur.

Bir bileşen koleksiyon olduğunda kompakt kurucuya her seferinde `List.copyOf` yazın.

Enum'lar

Bir enum tam sayı sabitleri listesi değildir. Her sabit, kendi durumu ve isterseniz kendi davranışı olan tam bir nesnedir. `Enums.java` dosyasından:

```
MERCURY  gravity  3.70   a 75kg person weighs  277.7 N
EARTH    gravity  9.80   a 75kg person weighs  735.2 N
MARS     gravity  3.71   a 75kg person weighs  278.4 N
```

Sabite özgü davranış bilinmeye değer özelliktir:

```
enum Operation {
    PLUS("+") { @Override double apply(double a, double b) { return a + b; } },
    MINUS("-") { @Override double apply(double a, double b) { return a - b; } };

    abstract double apply(double a, double b);
}
```

`apply` metodu olmadan sabit eklemek derleme hatasıdır. Alternatif, başka bir yerdeki bir `switch`, elle bulunup güncellenmek zorundadır.

`ordinal()` bir tuzaktır. Bildirim konumudur, dolayısıyla araya sabit eklemek sonrasındaki her şeyi sessizce yeniden numaralandırır:

```
PENDING -> stored as P, ordinal happens to be 0
ACTIVE  -> stored as A, ordinal happens to be 1
FAILED  -> stored as X, ordinal happens to be 2
```

Bir ordinal değerini asla kalıcı olarak saklamayın ve asla onun üzerinde switch yapmayın. Sabite, `Status` örneğindeki gibi açık bir kod alanı verin.

Enum'lar, her sabit tekil olduğu için `==` kullanımının deyimsel olduğu tek yerdir.

Biçimi seçmek

Kullanın	Ne zaman
<code>record</code>	tip verisinin kendisiyse ve aynı veriye sahip iki örnek aynı şeyse
<code>enum</code>	örnek kümesi sabit ve derleme zamanında biliniyorsa
<code>class</code>	kimliği, değiştirilebilir durumu ya da verisinden ağır basan davranışı varsa

Karani veren soru: **iki örnek aynı değerleri tutuyorsa birbirinin yerine geçer mi?** İki `Point(3, 4)` geçer. Aynı bakiyeye sahip iki banka hesabı geçmez.

Çalıştırın

```
java modules/08-records-and-enums/examples/Records.java
java modules/08-records-and-enums/examples/CompactConstructors.java
java modules/08-records-and-enums/examples/ShallowImmutability.java
java modules/08-records-and-enums/examples/Enums.java

./scripts/verify-examples.sh modules/08-records-and-enums
```

Sık Yapılan Hatalar

Bir record'a değiştirilebilir koleksiyon koyup ona değişmez demek. İki sızıntı, tek satır düzeltme.

`ordinal()` değerini kalıcı saklamak. Biri araya sabit ekler ve saklanmış her satır artık başka bir şey demektir.

Kimliği olan bir şey için record kullanmak. Bir `User` record'u, aynı ad ve e-postaya sahip iki kullanıcının aynı kullanıcı olduğu anlamına gelir. Bazen doğru, genellikle değil.

Normalleştirmeyi unutmak. Ham bileşenleri karşılaştıran bir record, `6/8` ile `3/4` değerlerini farklı kesirler sayar.

Kompakt kurucuda `this.x = x` yazmak. Derlenmez. Parametreye atayın.

Bir enum üzerinde beş ayrı yerde `switch` yazmak. Sabite özgü davranış mantığı veriyle birlikte tutar ve bir eksiği derleme hatasına çevirir.

Özet Çıkarımlar

- **Bir record bileşenlerinin ta kendisidir.** Derleyici `equals`, `hashCode`, `toString` ve erişimcileri doğru biçimde yazar.
- **Record'lar final'dır ve zaten `Record` sınıfını genişletir,** dolayısıyla kalıtım yok, ama arayüzler sorun değil.
- **Kompakt kurucular doğrular ve normalleştirir,** ve parametreye atarlar.
- **Değişmezlik sığdır.** Koleksiyon bileşenini her seferinde `List.copyOf` yapın.
- **Enum sabitleri nesnedir** ve durum ile sabite özgü davranış taşıyabilir.
- `ordinal()` değerini kalıcı hiçbir şey için kullanmayın.
- **Eşit veri eşit şey demekse record seçin,** örnekler sabitse enum, aksi hâlde sınıf.

Ödev

[homework/README.md](#)

Record'lar ve enum'larla küçük bir sipariş sistemi modelleyin, sonra bilerek yarattığınız iki değişmezlik sızıntısını kapatın. Referans çözüm [solutions/08-records-and-enums/](#) altında.